Mini App - Focus App - ADHD-ONT

Product Design Document

Table of Contents

- 1. Purpose
    - 1.1 General Requirements
- 2. Product Architecture
    - 2.1 Actors and Use-Cases
    - 2.2 Frontend Requirements
    - 2.3 API Requirements
    - 2.4 Backend Requirements

1. Purpose / Product Perspective

“Focus-type” apps exist on the market, yet are often pay-walled with microtransactions or not a free download. The MVP is to create a free similar in-design web application that allows its users to set a determined time threshold to “lockdown” non-essential websites on their computer.

References

Focus App PRD/SRS - Darien | Daveigh | King

1.1 General Requirements

The current specification pertains to the existing MVP defined in 1. Purpose.

Functional Requirements

1. The application should let users be able to specify a threshold of time.
2. The application should let users specify which web domains to lockdown.
3. The application should let users log in and log out of the web application.
4. The application should only provide a function when logged in.
5. The application should provide a default list of web domains to lockdown.
6. The application should lockdown non-essential web domains for a specified threshold of time.
7. The application should not feature microtransactions or be paid for.
8. The application should store user session data.
9. The application should store the threshold of time.
10. The application should alert the user when the threshold of time has ended.
11. The application should allow the user to stop the threshold of time earlier.
12. The application should have access to only know what other websites the user has open.
13. The application should award the user in-app currency for the amount of time threshold completed.
14. The application should stay loaded in the browser at all times until closed.
15. The application should use a browser extension to perform domain restriction.

Non-Functional Requirements / Constraints

1. The application should be deployed on a Web Browser.
2. The application should not support mobile browsers.

3. The application should not lock time limit thresholds behind paywalls.
4. The application should be completed by November 7th, 2026.
5. The application should utilize a database and server.
6. The application should manage data security or privacy.
7. The application should support account-based operations.
8. The application should use less than 500-1000 MB of RAM.
9. The application should support Google Chrome, Safari, and Firefox.

Assumptions and Dependencies

1. The application should depend on existing languages and frameworks.
2. The application should assume a developed frontend in a supported programming language.
3. The application should assume a developed backend in a supported programming language.
4. The application should assume a developed database in a supported programming language.

2. Product Architecture

2.1 Actors and Use-Cases

1. Actor - Users: General individuals who are utilizing the Web Application to restrict specific web domains for a specific period of time.
2. Use Case - Modify the Block-List
3. Use Case - Set the Time Threshold
4. Use Case - Stop the Time Threshold
5. Use Case - Login to User Account

6. Use Case - Logout of User Account
7. Use Case - Register User Account

2.2 Frontend Requirements

1. The application should provide button-based operations for the above use-cases.
2. The application should support button subtle border, pop, and gradient animations.
3. The application should support an animated background, a gently moving gradient.
4. The application should support the Manrope font.
5. The application should utilize a palette of “calming” colors. Below are a few examples, but not limited to:
    a. **Hex color code: #ABD7EB**
    b. **Hex color code: #6B9797**
    c. **Hex color code: #87CEEB**
6. The application should support dark and light mode in browsers.
7. The application should use REST API + Fastify
8. The application should be deployed using Vercel + Railway + Supabase.

2.3 API Requirements

1. The application should provide communication between the front-end operations and back-end operations.
2. The application should provide appropriate API interactions for each button-based operation.
3. The application should use Node.js + TypeScript + Fastify

2.4 Backend Requirements

1. The application should support data storage per user using the database.
2. The application should provide data encryption for sensitive user information.
3. The application should record in-app currency per user using the database.
4. The application should record domain blocklists per user using the database.
5. The application should record username and password per user using the database.
6. The application should register new users in the database.
7. The application should associate data and users in the database.
8. The application should use React + TypeScript + Vite
9. The application should use WXT + TypeScript for its Web Extension.

2.5 Database Requirements

1. The application should keep records of all relevant data specified by the backend requirements.
2. The application should use PostgreSQL (Supabase)